

- zawieranie się, gromadzenie elementów – nowa klasa powstaje przez użycie obiektów klas już istniejących,
- klasa może być zbudowana z dowolnej liczby obiektów
 - obiekty te mogą być dowolnych typów (również kolekcje)
- często mówimy o relacji zawierania np. „ciało zawiera rękę” - gdzie Ręka i Ciało są klasami, oraz klasa Ciało zawiera w sobie obiekt (a w zasadzie 2 obiekty) typu Ręka.
- Agregacja nie tworzy klasy pochodnej, lecz zupełnie nowy typ.

Agregacja - prosty przykład

Programowanie
obiektywne

Agregacja,
kompozycja

Agregacja
Kompozycja

Pakiety i
moduły

Moduły
Pakiety

Serializacja
pickle

Wzorce
projektowe
Wzorzec Iterator

Testowanie
przy OOP
Czemu testować
Czemu testować

Idiomy
Pythona
for
enumerate
izip

```
class Reka:
    def __init__(self, ktora):
        self.ktora = ktora

class Noga:
    def __init__(self, ktora):
        self.ktora = ktora

class CialoSamo:
    def __init__(self):
        self.lewa_reka = Reka("LEWA")
        self.prawa_reka = Reka("PRAWA")
        self.lewa_noga = Noga("LEWA")
        self.prawa_noga = Noga("PRAWA")

x = CialoSamo()
```

Agregacja - prosty przykład

Programowanie obiektowe

Agregacja, kompozycja

Agregacja
Kompozycja

Pakiety i moduły

Moduły
Pakiety

Serializacja

pickle

Wzorce projektowe

Wzorzec Iterator

Testowanie przy OOP

Czemu testować
Czemu testować

Idiomy Pythona

for
enumerate
izip

```
class Reka:
    def __init__(self, ktora):
        self.ktora = ktora

class Noga:
    def __init__(self, ktora):
        self.ktora = ktora

class Cialo:
    def __init__(self, lewa_reka, prawa_reka,
                 lewa_noga, prawa_noga):
        self.lewa_reka = lewa_reka
        self.prawa_reka = prawa_reka
        self.lewa_noga = lewa_noga
        self.prawa_noga = prawa_noga
```

cdn.

Agregacja - prosty przykład cd.

Programowanie obiektowe

Agregacja, kompozycja

Agregacja Kompozycja

Pakiety i moduły

Moduły Pakiety

Serializacja

pickle

Wzorce projektowe

Wzorzec Iterator

Testowanie przy OOP

Czemu testować Czemu testować

Idiomy Pythona

for enumerate izip

```
moja_lewa_reka = Reka("LEWA")
moja_prawa_reka = Reka("PRAWA")
moja_lewa_noga = Noga("LEWA")
moja_prawa_noga = Noga("PRAWA")

x= Cialo(moja_lewa_reka , moja_prawa_reka ,
        moja_lewa_noga , moja_prawa_noga)
```

Agregacja - bardziej realny przykład

```
class Punkt:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def pokaz(self):
        print("X: " + str(self.x) + "Y: " + str(
            self.y))

class Odcinek:
    def __init__(self, punkt_początkowy,
        punkt_koncowy):
        self.punkt_początkowy = punkt_początkowy
        self.punkt_koncowy = punkt_koncowy

    def pokaz(self):
        print("Punkt początkowy:", end = ' ')
        self.punkt_początkowy.pokaz()
        print("Punkt końcowy:", end = ' ')
        self.punkt_koncowy.pokaz()
```

Agregacja - bardziej realny przykład

Programowanie
obiektywne

Agregacja,
kompozycja

Agregacja
Kompozycja

Pakiety i
moduły

Moduły
Pakiety

Serializacja
pickle

Wzorce
projektowe

Wzorec Iterator

Testowanie
przy OOP

Czemu testować
Czemu testować

Idiomy
Pythona

for
enumerate
izip

Tworzenie obiektów, obiekty nazwane jako parametry,

```
poczatek = Punkt(10, 15)
koniec = Punkt(20, 35)
odcinek = Odcinek(poczatek, koniec)
odcinek.pokaz()
```

Wynik:

```
Punkt poczatkowy: X: 10 Y :15
Punkt koncowy: X: 20 Y :35
```



Agregacja - bardziej realny przykład

Programowanie
obiektywne

Agregacja,
kompozycja

Agregacja
Kompozycja

Pakiety i
moduły

Moduły
Pakiety

Serializacja
pickle

Wzorce
projektowe
Wzorzec Iterator

Testowanie
przy OOP
Czemu testować
Czemu testować

Idiomy
Pythona
for
enumerate
izip

Tworzenie obiektów, obiekty jako parametry tworzone w trakcie konstruowania agregatu:

```
odcinek = Odcinek(Punkt(10, 15), Punkt(20, 35))  
odcinek.pokaz()
```

Wynik:

```
Punkt poczatkowy: X: 10   Y :15  
Punkt koncowy: X: 20   Y :35
```

- Główna różnica polega na tym, że obiekty należące do obiektu agregującego nie mogą istnieć (nie ma sensu ich istnienie) poza obiektem grupującym.
 - różnica jest niezbyt istotna
- w poprzednich przykładach możemy mówić o kompozycji (ang. composition) o ile Ręki czy Punktu nie używamy na zewnątrz,
- jeżeli obiekty składowe nie są tworzone na zewnątrz klasy to klasy dla tych obiektów można zapisać wewnątrz metody klasy agregującej (np. w `__init__`) lub w samej klasie.

Definicja klas w klasie, trzeba się odpowiednio odwoływać:

```
class CialoSamo:
    class Reka:
        def __init__(self, ktora):
            self.ktora = ktora
    class Noga:
        def __init__(self, ktora):
            self.ktora = ktora
    def __init__(self):
        self.lewa_reka = CialoSamo.Reka("LEWA")
        self.prawa_reka = CialoSamo.Reka("PRAWA")
        self.lewa_noga = CialoSamo.Noga("LEWA")
        self.prawa_noga = CialoSamo.Noga("PRAWA")

x = CialoSamo()
```

Kompozycja - przykład

Definicja klas w metodzie, krótszy zapis, przy większych przypadkach może być jednak mało czytelne:

```
class CialoSamo:
    def __init__(self):
        class Reka:
            def __init__(self, ktora):
                self.ktora = ktora

        class Noga:
            def __init__(self, ktora):
                self.ktora = ktora

        self.lewa_reka = Reka("LEWA")
        self.prawa_reka = Reka("PRAWA")
        self.lewa_noga = Noga("LEWA")
        self.prawa_noga = Noga("PRAWA")
```

```
x = CialoSamo()
```

Większy program można (od pewnego momentu wręcz należy) podzielić na moduły:

- w module możemy umieścić pojedynczą klasę lub funkcję,
- możemy również umieszczać wiele klas i funkcji,
- podstawowym kryterium podziału powinna być jego logika
 - np. nie łączymy funkcji geometrycznych do rysowania figur z klasami do zarządzania osobami
 - np. klasę bazową można umieszczać wspólnie z jej klasami pochodnymi,
- nazwy modułów powinny być raczej krótkie, zgodne z zawartością.



Moduły - przykłady

Programowanie
obiektowe

Agregacja,
kompozycja

Agregacja
Kompozycja

Pakiety i
moduły

Moduły
Pakiety

Serializacja
pickle

Wzorce
projektowe
Wzorzec Iterator

Testowanie
przy OOP
Czemu testować
Czemu testować

Idiomy
Pythona
for
enumerate
izip

Plik osobaModule.py - jedna klasa w module

```
class Osoba:
    def __init__(self, imie, nazwisko):
        self.imie = imie
        self.nazwisko = nazwisko

    def pokaz(self):
        print(self.imie + ' ' + self.nazwisko)
```



Moduły - przykłady

Programowanie
obiektowe

Agregacja,
kompozycja

Agregacja
Kompozycja

Pakiety i
moduły

Moduły
Pakiety

Serializacja
pickle

Wzorce
projektowe
Wzorzec Iterator

Testowanie
przy OOP

Czemu testować
Czemu testować

Idiomy
Pythona

for
enumerate
zip

Plik osobaModule.py - jedna klasa w module

```
>>> import OsobaModule
Traceback (most recent call last):
  File "<pyshell#0>", line 1, in <module>
    import OsobaModule
ImportError: No module named 'OsobaModule'
```

```
>>> import osobaModule
```

```
>>> from osobaModule
SyntaxError: invalid syntax
```

```
>>> from osobaModule import Osoba
```

Plik osobaModule.py - jedna klasa w module

```
>>> import osobaModule
>>> x = Osoba("Jan", "Nowak")
Traceback (most recent call last):
  File "<pyshell#7>", line 1, in <module>
    x = Osoba("Jan", "Nowak")
NameError: name 'Osoba' is not defined
```

```
>>> import osobaModule
>>> x = osobaModule.Osoba("Jan", "Nowak")
>>> x.pokaz()
Jan Nowak
```

```
>>> from osobaModule import Osoba
>>> x = osobaModule.Osoba("Jan", "Nowak")
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
```



Moduły - przykłady

Programowanie
obiektowe

Agregacja,
kompozycja

Agregacja
Kompozycja

Pakiety i
moduły

Moduły
Pakiety

Serializacja
pickle

Wzorce
projektowe
Wzorec Iterator

Testowanie
przy OOP
Czemu testować
Czemu testować

Idiomy
Pythona
for
enumerate
izip

Plik osobaModule.py - jedna klasa w module

```
>>> from osobaModule import Osoba
      x = osobaModule.Osoba("Jan", "Nowak")
NameError: name 'osobaModule' is not defined
```

```
>>> from osobaModule import Osoba
>>> x = Osoba("Jan", "Nowak")
>>> x.pokaz()
Jan Nowak
>>>
```

Plik osobaModule.py - jedna klasa w module

```
>>> from osobaModule import *
>>> x = osobaModule.Osoba("Jan", "Nowak")
Traceback (most recent call last):
  File "<pyshell#6>", line 1, in <module>
```

```
>>> from osobaModule import *
x = osobaModule.Osoba("Jan", "Nowak")
NameError: name 'osobaModule' is not defined
```

```
>>> from osobaModule import *
>>> x = Osoba("Jan", "Nowak")
>>> x.pokaz()
Jan Nowak
```

Plik osobaModule.py - dwie klasy w module

```
class Osoba:
    def __init__(self, imie, nazwisko):
        self.imie = imie
        self.nazwisko = nazwisko

    def pokaz(self):
        print(self.imie + ' ' + self.nazwisko)

class Student(Osoba):
    def __init__(self, imie, nazwisko, nr_indeksu):
        super().__init__(imie, nazwisko)
        self.nr_indeksu = nr_indeksu

    def pokaz(self):
        super().pokaz()
        print(self.nr_indeksu)
```



Moduły - przykłady

Programowanie
obiektowe

Agregacja,
kompozycja

Agregacja
Kompozycja

Pakiety i
moduły

Moduły
Pakiety

Serializacja
pickle

Wzorce
projektowe
Wzorec Iterator

Testowanie
przy OOP
Czemu testować
Czemu testować

Idiomy
Pythona
for
enumerate
izip

Plik osobaModule.py - dwie klasy w module

```
>>> from osobaModule import Osoba, Student
>>> x=Student("Jan", "Nowak", 1234)
>>> x.pokaz()
Jan Nowak
1234
```

Moduły - przykłady

Programowanie
obiektywne

Agregacja,
kompozycja

Agregacja
Kompozycja

Pakiety i
moduły

Moduły
Pakiety

Serializacja
pikle

Wzorce
projektowe

Wzorzec Iterator

Testowanie
przy OOP

Czemu testować
Czemu testować

Idiomy
Pythona

for
enumerate
izip

Plik osobaModule.py - dwie klasy w module, inne zachowanie niż w niektórych językach.

```
>>> from osobaModule import Student
>>> x = Student("Jan", "Nowak", 1234)
>>> z = Osoba("Andrzej", "Kowalski")
Traceback (most recent call last):
  File "<pyshell#2>", line 1, in <module>
    z = Osoba("Andrzej", "Kowalski")
NameError: name 'Osoba' is not defined
```

```
>>> from osobaModule import Student
>>> x = Student("Jan", "Nowak", 1234)
>>> x.pokaz()
Jan Nowak
1234
```

Możliwość lokalnego korzystania z innej nazwy dla importowanego modułu lub klasy:

```
import nazwa_modulu as lokalna_nazwa_modulu
from nazwa_modulu import nazwa_klasy as
    lokalna_nazwa_klasy
```

Dla klasy:

```
>>> from osobaModule import Osoba as Person
>>> x=Person("Andrzej", "Abacki")
>>> x
<osobaModule.Osoba object at 0x029FDE90>
>>> x.__dict__
{'imie': 'Andrzej', 'nazwisko': 'Abacki'}
```

Możliwość lokalnego korzystania z innej nazwy dla importowanego modułu lub klasy:

```
import nazwa_modulu as lokalna_nazwa_modulu
from nazwa_modulu import nazwa_klasy as
    lokalna_nazwa_klasy
```

Dla modułu:

```
>>> import osobaModule as omod
>>> x=omod.Osoba("Andrzej", "Abacki")
>>> x
<osobaModule.Osoba object at 0x0278FDF0>
```

Do grupowania modułów

- tak naprawdę kolejny poziom grupowania (abstrakcji),
- moduły też można zagnieżdżać,
- Pakiet może zawierać jeden moduł,
- ale raczej powinien zawierać pewien zbiór modułów i innych rzeczy,
- **Pakiet** to kolekcja modułów w katalogu,
- nazwa pakietu to nazwa tego katalogu,
- żeby można było importować moduły z pakietu, trzeba w nim umieścić (wystarczy pusty) plik `__init__.py`

Pakiety - przykład

Pakiet o nazwie obiekty:

```

obiekty
|   Cialo.py
|   CialoSamo.py
|   __init__.py
|
+--- geometria
|   Punkty.py
|   __init__.py
|
\--- osoby
     osobaModule.py
     __init__.py
    
```

Ciekawostka, powyższe uzyskać można poleceniem DOS:

```
tree obiekty /A /F
```

Z zewnątrz pakietu (bezwzględny):

```
import nazwa_pakietu.nazwa_modulu
x = nazwa_pakietu.nazwa_modulu.NazwaKlasy()
```

```
from nazwa_pakietu.nazwa_modulu import NazwaKlasy
x = NazwaKlasy()
```

```
from nazwa_pakietu import nazwa_modulu
x = nazwa_modulu.NazwaKlasy()
```



Pakiety - importowanie

Programowanie
obiektowe

Agregacja,
kompozycja

Agregacja
Kompozycja

Pakiety i
moduły

Moduły

Pakiety

Serializacja
pickle

Wzorce
projektowe

Wzorzec Iterator

Testowanie
przy OOP

Czemu testować

Czemu testować

Idiomy
Pythona

for

enumerate

izip

Źle:

```
>>> from obiekty import CialoSamo
>>> x=CialoSamo()
Traceback (most recent call last):
  File "<pyshell#24>", line 1, in <module>
    x=CialoSamo()
TypeError: 'module' object is not callable
```

Dobrze:

```
>>> from obiekty.CialoSamo import CialoSamo
>>> x=CialoSamo()
>>> x
<obiekty.CialoSamo.CialoSamo object at 0x02A10CB0>
```

Pakiety - importowanie

Programowanie
obiektywne

Agregacja,
kompozycja

Agregacja
Kompozycja

Pakiety i
moduły

Moduły

Pakiety

Serializacja
pickle

Wzorce
projektowe

Wzorzec Iterator

Testowanie
przy OOP

Czemu testować

Czemu testować

Idiomy
Pythona

for

enumerate

izip

Dobrze, ale trochę dużo pisania:

```
from obiekty.geometria.Punkty import Odcinek, Punkt
>>> odcinek = Odcinek(Punkt(10, 15), Punkt(20, 35))
>>> odcinek.pokaz()
Punkt poczatkowy: X: 10 Y :15
Punkt koncowy: X: 20 Y :35
<obiekty.CialoSamo.CialoSamo object at 0x026199F0>
```

Skąd ten obiekt na końcu?

Pakiety - importowanie

Programowanie
obiektywne

Agregacja,
kompozycja

Agregacja
Kompozycja

Pakiety i
moduły

Moduły

Pakiety

Serializacja

pickle

Wzorce
projektowe

Wzorzec Iterator

Testowanie
przy OOP

Czemu testować

Czemu testować

Idiomy
Pythona

for

enumerate

izip

Z wnętrza pakietu (względny). Z pakietu z tego samego katalogu

```
from .nazwa_modulu import NazwaKlasy  
x = NazwaKlasy()
```

Z pakietu z katalogu wyżej (z pakietu nadrzędnego):

```
from ..nazwa_modulu import NazwaKlasy  
x = NazwaKlasy()
```

```
from ..nazwa_pakietu.nazwa_modulu import NazwaKlasy  
x = NazwaKlasy()
```



Pakiety - importowanie

Programowanie
obiektywne

Agregacja,
kompozycja

Agregacja
Kompozycja

Pakiety i
moduły

Moduły
Pakiety

Serializacja
pickle

Wzorce
projektowe
Wzorzec Iterator

Testowanie
przy OOP
Czemu testować
Czemu testować

Idiomy
Pythona
for
enumerate
izip

Jesteśmy w pliku osobaModule.py Dobrze:

```
from .osobaModule import Osoba
```

Jesteśmy w pliku Punkty.py

```
from ..CialoSamo import CialoSamo
```



Zapis obiektów do pliku

Programowanie obiektowe

Agregacja, kompozycja

Agregacja
Kompozycja

Pakiety i moduły

Moduły
Pakiety

Serializacja pickle

Wzorce projektowe

Wzorzec Iterator

Testowanie przy OOP

Czemu testować
Czemu testować

Idiomy Pythona

for
enumerate
zip

- Python oferuje moduł pickle
 - Skojarzenie ze słoikiem ogórków prawidłowe.
- pickle zamienia obiekt i wszystkie obiekty będące jego atrybutami w ciąg bajtów
 - może być on przechowany,
 - lub przesłany.

```
class CialoSamo:
    class Reka:
        def __init__(self, ktora):
            self.ktora = ktora
    class Noga:
        def __init__(self, ktora):
            self.ktora = ktora
    def __init__(self):
        self.lewa_reka = CialoSamo.Reka("LEWA")
        self.prawa_reka = CialoSamo.Reka("PRAWA")
        self.lewa_noga = CialoSamo.Noga("LEWA")
        self.prawa_noga = CialoSamo.Noga("PRAWA")
```

cdn.

Serializacja - przykład

Programowanie
obiektowe

Agregacja,
kompozycja

Agregacja
Kompozycja

Pakiety i
moduły

Moduły
Pakiety

Serializacja
pickle

Wzorce
projektowe
Wzorzec Iterator

Testowanie
przy OOP
Czemu testować
Czemu testować

Idiomy
Pythona
for
enumerate
izip

Przy okazji dobry sposób na współpracę z plikiem:

```
x = CialoSamo()  
import pickle  
with open("cialo.bin", 'wb') as file:  
    pickle.dump(x, file)  
with open("cialo.bin", 'rb') as file:  
    z = pickle.load(file)  
print(z.lewa_reka.ktora)
```

Wynik (zapisaliśmy obiekt x do pliku i wczytaliśmy go jako z):

LEWA

Serializacja - przykład

Programowanie
obiektywne

Agregacja,
kompozycja

Agregacja
Kompozycja

Pakiety i
moduły

Moduły
Pakiety

Serializacja
pickle

Wzorce
projektowe
Wzorzec Iterator

Testowanie
przy OOP

Czemu testować
Czemu testować

Idiomy
Pythona

for
enumerate
izip

Wersja z przesyłaniem ciągu bajtów:

```
x = CialoSamo()
import pickle

zrzut_bajtowy = pickle.dumps(x)
y = pickle.loads(zrzut_bajtowy)
print(y.lewa_reka.ktora)
```

Wynik (zapisaliśmy obiekt x do ciągu bajtów i zamieniliśmy go (ciąg bajtów) na y):

LEWA



Wzorce projektowe

Programowanie
obiektowe

Agregacja,
kompozycja

Agregacja
Kompozycja

Pakiety i
moduły

Moduły
Pakiety

Serializacja
pickle

Wzorce
projektowe

Wzorzec Iterator

Testowanie
przy OOP

Czemu testować
Czemu testować

Idiomy
Pythona

for
enumerate
izip

- Wiele problemów ma już gotowe rozwiązania,
- Często nasz problem jest podobny, można go podobnie rozwiązać jak problem rozwiązany przez kogoś innego
- Każdy wzorzec projektowy proponuje zestaw obiektów oddziałujących w specyficzny sposób w celu rozwiązania ogólnego problemu.

- Iterator to obiekt który ma dwie metody: next i done
 - next podaje następny element,
 - done sprawdza, czy już skończyły się elementy

```
while not elementy.done():  
    element = elementy.next()  
    #zrob cos z element
```

W Pythonie zawsze można prościej:

```
for element in elementy:  
    #zrob cos z element
```

```
class PangramIterable:
    def __init__(self, pangram):
        self.pangram = pangram
    def __iter__(self):
        return PangramIterator(self.pangram)

class PangramIterator:
    def __init__(self, pangram):
        self.slowa = [s.capitalize() for s in
                      pangram.split()]
        self.index = 0
    def __next__(self):
        if self.index == len(self.slowa):
            raise StopIteration()
        slowo = self.slowa[self.index]
        self.index += 1
        return slowo
    def __iter__(self):
        return self
```

Forma dłuższa:

```

pangram_iterable = PangramIterable('the_quick_brown_
fox_jumps_over_the_lazy_dog')
iterator = iter(pangram_iterable)
while True:
    try:
        print(next(iterator))
    except StopIteration:
        break

```

Forma krótsza: pangram_iterable = PangramIterable('Mężny
bądź, chroń pułk twój i sześć flag')

```

for i in pangram_iterable:
    print(i)

```



Testowanie - czemu to ważne

Programowanie
obiektowe

Agregacja,
kompozycja

Agregacja
Kompozycja

Pakiety i
moduły

Moduły
Pakiety

Serializacja
pickle

Wzorce
projektowe
Wzorzec Iterator

Testowanie
przy OOP
Czemu testować
Czemu testować

Idiomy
Pythona
for
enumerate
izip

- niektórzy twierdzą, że to ze względu na dynamiczną naturę (wszystko można zmienić) testowanie w Pythonie jest jeszcze ważniejsze niż w innych językach,
 - ale testy tak naprawdę nie sprawdzają typów tylko wartości,
- to dlaczego w Pythonie pisze się tak dużo testów?
 - bo to w nim bardzo łatwe.
- Testujemy po to, żeby się upewnić, że kod działa prawidłowo.



Testowanie - czemu to ważne

Programowanie
obiektowe

Agregacja,
kompozycja

Agregacja
Kompozycja

Pakiety i
moduły

Moduły
Pakiety

Serializacja
pickle

Wzorce
projektowe
Wzorzec Iterator

Testowanie
przy OOP
Czemu testować
Czemu testować

Idiomy
Pythona
for
enumerate
izip

- Ponieważ przy poprawkach możemy zepsuć inną rzecz niż tą którą naprawialiśmy, warto testy zautomatyzować.
- przy większych programach testowanie ręczne całości programu jest praktycznie niemożliwe.
- w okolicach 2000 roku spopularyzowały się pojęcia TDD (Test Driven Development - Programowanie Napędzane Testami ;)) i Unit Tests (testy jednostkowe).

```
class Konto:
    def __init__(self, numer, kwota):
        self.numer=numer
        self.kwota = kwota
    def wyplac(self, wartosc):
        self.kwota -= wartosc
    def wplac(self, wartosc):
        self.kwota += wartosc #celowa pomyłka
    def przelew(self, docelowe, kwota):
        self.wyplac(kwota)
        docelowe.wplac(kwota)
```

```
import unittest
class KontoTest(unittest.TestCase):
    def test_przelew_prosty(self):
        konto_glowne = Konto("1234", 100)
        konto_dodatkowe = Konto("1235", 200)
        konto_glowne.przelew(konto_dodatkowe, 50)
        self.assertEqual(konto_glowne.kwota, 50)
        self.assertEqual(konto_dodatkowe.kwota, 250)

    def test_przelew_zlozony(self):
        konto_glowne = Konto("1234", 100)
        konto_dodatkowe = Konto("1235", 200)
        konto_glowne.przelew(konto_dodatkowe, 50)
        konto_dodatkowe.przelew(konto_glowne, 150)
        self.assertEqual(konto_glowne.kwota, 200)
        self.assertEqual(konto_dodatkowe.kwota, 100)

if __name__ == "__main__":
    unittest.main()
```



Testy - przykład

Programowanie
obiektywne

Agregacja,
kompozycja

Agregacja
Kompozycja

Pakiety i
moduły

Moduły
Pakiety

Serializacja
pickle

Wzorce
projektowe
Wzorzec Iterator

Testowanie
przy OOP
Czemu testować
Czemu testować

Idiomy
Pythona

for
enumerate
zip

FF

FAIL: test_przelew_prosty (__main__.KontaTest)

Traceback (most recent call last):

File "kontaTesty.py", line 19, in

test_przelew_prosty

self.assertEqual(konto_dodatkowe.kwota, 250)

AssertionError: 150 != 250

FAIL: test_przelew_zlozony (__main__.KontaTest)

Traceback (most recent call last):

File "kontaTesty.py", line 26, in

test_przelew_zlozony

self.assertEqual(konto_glowne.kwota, 200)

AssertionError: -100 != 200

Ran 2 tests in 0.005s

FAILED (failures=2)



for

Programowanie
obiektowe

Agregacja,
kompozycja

Agregacja
Kompozycja

Pakiety i
moduły

Moduły
Pakiety

Serializacja
pickle

Wzorce
projektowe
Wzorzec Iterator

Testowanie
przy OOP

Czemu testować
Czemu testować

Idiomy
Pythona

for
enumerate
zip

Zamiast:

```
elements = [(1, 2), (3,4), (8,9)]  
for i in range(len(elements):  
    print(elements[i])
```

Lepiej napisać:

```
elements = [(1, 2), (3,4), (8,9)]  
for element in elements:  
    print(element)
```



enumerate

Programowanie
obiektowe

Agregacja,
kompozycja

Agregacja
Kompozycja

Pakiety i
moduły

Moduły
Pakiety

Serializacja
pickle

Wzorce
projektowe
Wzorzec Iterator

Testowanie
przy OOP

Czemu testować
Czemu testować

Idiomy
Pythona

for
enumerate
izip

Zamiast:

```
elements = [(1, 2), (3,4), (8,9)]  
for i in range(len(elements)):  
    print(i, elements[i])
```

Lepiej napisać:

```
elements = [(1, 2), (3,4), (8,9)]  
for i, element in enumerate(elements):  
    print(i, element)
```

```
color_hex = ["0xFF0000", "0x00FF00", "0x0000FF"]
color_names = ["red", "green", "blue"]
```

Zamiast:

```
length = min(len(color_hex), len(color_names))
colors={}
for i in range():
    colors[color_names[i]] = color_hex[i]
for k, v in colors.items():
    print(k, v)
```

Lepiej napisać:

```
colors = zip(color_names, color_hex)
for k, v in colors:
    print(k, v)
```

```
color_hex = ["0xFF0000", "0x00FF00", "0x0000FF"]
color_names = ["red", "green", "blue"]
```

Zamiast:

```
length = min(len(color_hex), len(color_names))
colors={}
for i in range():
    colors[color_names[i]] = color_hex[i]
for k, v in colors.items():
    print(k, v)
```

Lepiej napisać (najczęściej nie musimy takiej wersji używać):

```
colors={}
for name, hex_value in zip(color_names, color_hex):
    colors[name] = hex_value
for k, v in colors.items():
    print(k, v)
```